Project - High Level Design

on

Haven AI

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Rudra Sthapak	EN22CS301828
2.	Rajvardhan Singh Rathore	EN22CS301786
3.	Ronak Bhawsar	EN22CS301824
4.	Rupendra Singh Tomar	EN22CS301833
5.	Raj Kumawat	EN22CS301779

Group Name: 04D7

Project Number: DO-27

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	3
Chapter 2	System Design	4-7
	2.1. Application Design	4
	2.2. Process Flow	4
	2.3. Information Flow	5
	2.4. Components Design	6
	2.5. Key Design Considerations	6
	2.6. API Catalogue	7
Chapter 3	Data Design	8-9
	3.1. Data Model	8
	3.2. Data Access Mechanism	8
	3.3. Data Retention Policies	9
	3.4. Data Migration	9
Chapter 4	Interfaces	10
Chapter 5	State and Session Management	11
Chapter 6	Caching	12
Chapter 7	Non-Functional Requirements	13
	7.1. Security Aspects	13
	7.2. Performance Aspects	13
Chapter 8	References	14

1. Introduction

This document presents the High-Level Design (HLD) of the Real Estate Support & Intelligent Triage Agent System with a focus on DevOps and Infrastructure as Code (IaC). It outlines the overall system architecture, major components, data handling strategies, and deployment mechanisms. The system integrates AI-based query processing with automated infrastructure provisioning and containerized deployment.

1.1. Scope of the Document

This document covers the high-level architecture, system workflows, data design, and DevOps implementation using tools such as Docker and IaC (Terraform/Ansible). It defines how the system is structured, deployed, and managed in a scalable and automated manner.

1.2. Intended Audience

This document is intended for project evaluators, developers, DevOps engineers, and stakeholders who require an understanding of the system architecture and deployment approach.

1.3. System Overview

The system is an AI-powered real estate agent that processes user queries using natural language understanding and provides filtered property results. The system is enhanced with DevOps practices, where infrastructure is provisioned using IaC and the application is deployed using Docker containers.

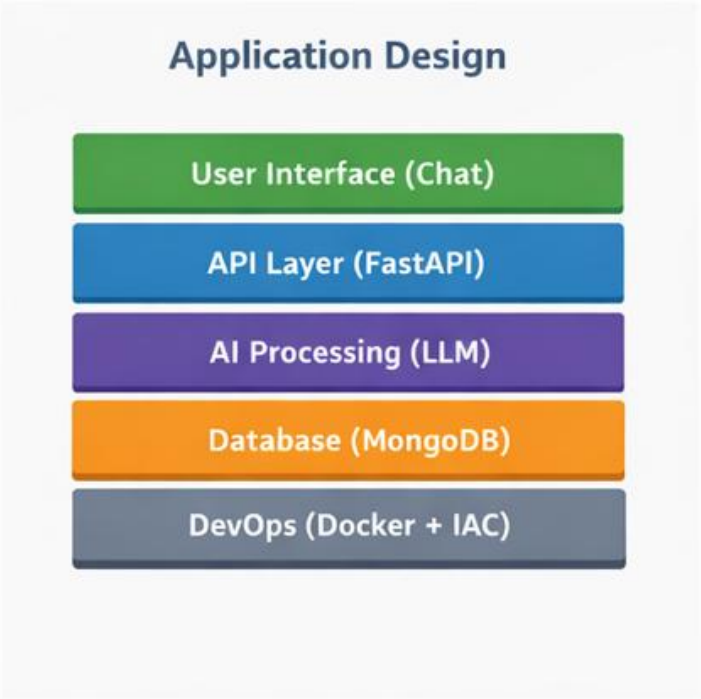
The architecture ensures automation, scalability, and consistency across environments.

2. System Design

2.1. Application Design

The system follows a layered architecture:

- **User Interface Layer** – Chat-based interaction
- **Backend Layer (FastAPI)** – API handling
- **AI Layer** – Query processing using LLM
- **Database Layer (MongoDB)** – Data storage
- **DevOps Layer** – Docker + IaC deployment

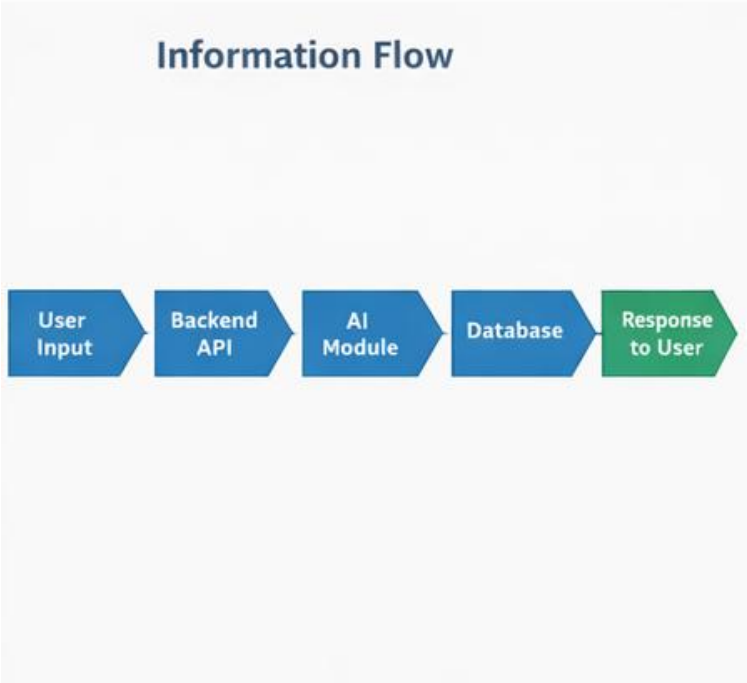


2.2. Process Flow

1. User sends query
2. Backend receives request
3. AI processes input
4. Entities extracted
5. Database queried
6. Results returned
7. Response displayed

2.3. Information Flow

User Input → Backend API → AI Module → Database ← Response returned to user



2.4. Components Design

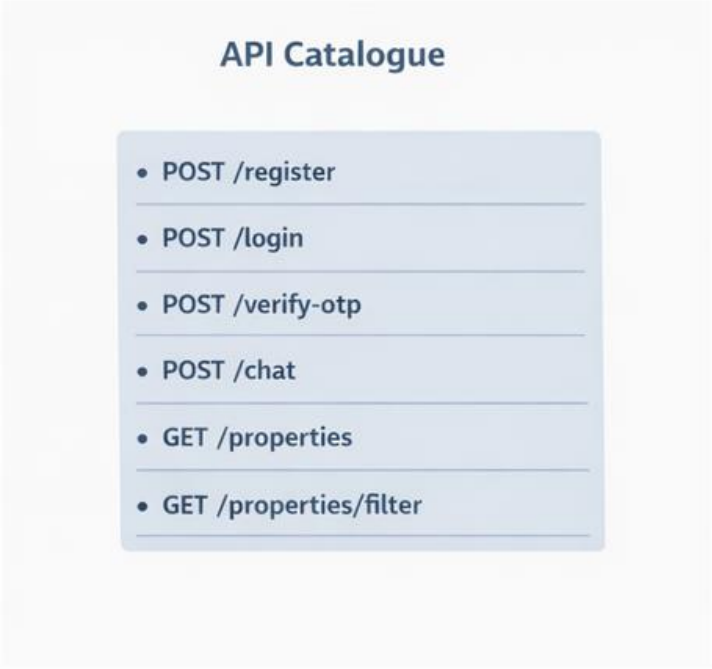
- Authentication Module
- Chat Processing Module
- AI Processing Module
- Property Search Module
- Database Module
- DevOps Module (Docker + IaC)

2.5. Key Design Considerations

- Scalability
- Security
- Performance
- Maintainability
- Reliability
- Automated Deployment (DevOps)

2.6. API Catalogue

- POST /register
- POST /login
- POST /verify-otp
- POST /chat
- GET /properties
- GET /properties/filter



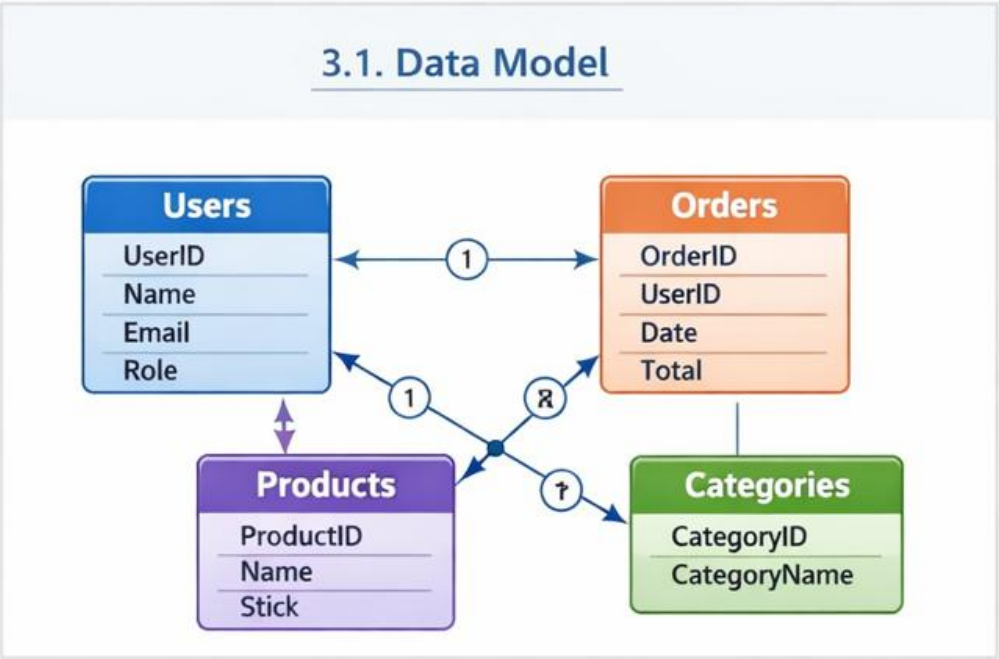
3. Data Design

3.1. Data Model

The system uses MongoDB collections:

- Users Collection
- Properties Collection
- OTP Collection

Each collection stores structured JSON documents for efficient querying.

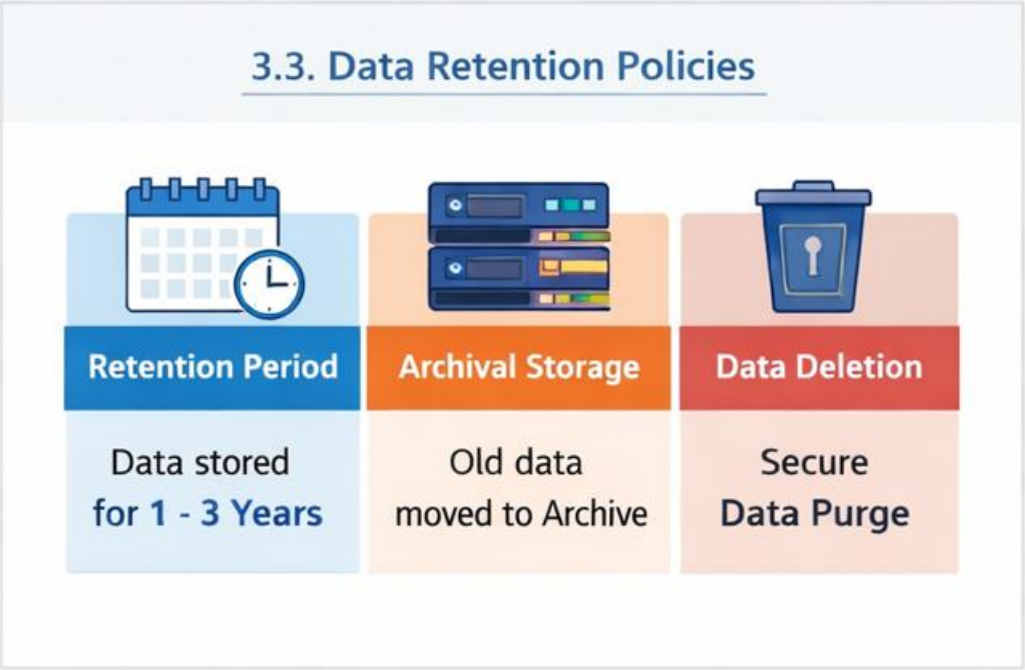


3.2. Data Access Mechanism

Data is accessed using asynchronous APIs through FastAPI. MongoDB queries are executed using filters based on user input such as location, price, and property type.

3.3. Data Retention Policies

- OTP data is temporary and expires automatically
- User data is stored securely
- Property data is retained for long-term use



3.4. Data Migration

Data migration can be performed using scripts to move data between environments (local to cloud). Backup and restore mechanisms ensure data integrity.

4. Interfaces

The system exposes well-defined RESTful interfaces to enable seamless communication between the frontend, backend, and external services. All interactions are performed through HTTP-based APIs, where requests and responses are exchanged in JSON format to ensure platform independence and ease of integration. The frontend chat interface communicates with the FastAPI backend using secure API endpoints for operations such as user authentication, query submission, and retrieval of property data. Additionally, the backend interfaces with the AI processing layer to send user queries and receive structured responses after intent recognition and entity extraction. These interfaces are designed to be scalable and loosely coupled, allowing future integration with third-party services such as cloud platforms or external property listing APIs without affecting the core system.

The Real Estate Support Triage Agent communicates through multiple well-defined interfaces that ensure modularity, scalability, and clear separation of responsibilities across system components. The primary interface exists between the frontend application and the backend API layer, where communication is handled using RESTful principles over HTTP/HTTPS. All requests and responses are exchanged in structured JSON format, ensuring standardized data transfer and easy extensibility. This interface remains stateless and follows proper HTTP status conventions to maintain consistency and reliability.

Beyond client-server communication, the backend interacts with the Large Language Model through an abstraction layer implemented using LangChain. This interface is responsible for prompt construction, entity extraction, and structured tool-calling. Instead of allowing the LLM to directly access the database, the system restricts it to predefined backend tools such as property search. This controlled interaction improves reliability, prevents unsafe operations, and ensures predictable system behavior.

The backend also communicates with the MongoDB database through an asynchronous data access layer using the Motor driver. This interface supports non-blocking operations, dynamic query construction, and indexed search mechanisms to ensure optimal performance. Internally, backend modules interact through clearly defined service-layer contracts, maintaining loose coupling and improving maintainability. Overall, the interface design ensures that each component of the system can evolve independently without disrupting other layers, which is essential for scalable and production-ready architecture.

5. State and Session Management

The system follows a stateless architecture, ensuring that each API request is processed independently without relying on server-side session storage. User authentication is handled through secure mechanisms such as OTP verification and token-based authentication, which allows the system to validate users without maintaining persistent session data on the server. This approach improves scalability, as multiple instances of the backend can handle requests without synchronization issues. Temporary data such as OTP codes are stored in the database with expiration policies, ensuring both security and efficiency. Additionally, conversational context for the AI module can be managed at the application level, allowing the system to provide context-aware responses while still maintaining overall statelessness in the API design.

The Real Estate Support Triage Agent follows a stateless architectural design, meaning that each request is processed independently without maintaining persistent server-side session storage. This approach simplifies horizontal scaling and reduces memory consumption on the server. Since no session state is stored between requests, the backend can efficiently handle concurrent users without synchronization issues.

Authentication is validated during each relevant request to ensure secure access to protected endpoints. Currently, chat interactions are processed independently, and conversation history is not persistently stored. This aligns with the project's defined scope and ensures lightweight backend processing. The stateless nature of the system also makes deployment and load balancing significantly easier.

However, the architecture is designed to support future enhancements such as JSON Web Token (JWT) based authentication. Token-based authentication would allow secure session validation without storing server-side state. Additionally, persistent chat context storage can be introduced to enable more personalized and context-aware conversations. Expiration policies, token refresh mechanisms, and inactivity timeout handling can be incorporated in future versions to strengthen both security and user experience while maintaining scalability.

6. Caching

Caching is implemented to improve system performance and reduce the load on the database by temporarily storing frequently accessed data. In the context of the real estate system, commonly searched property results and repeated queries can be cached to provide faster responses to users. Tools such as Redis can be integrated as an in-memory data store to enable high-speed data retrieval. Caching also helps minimize redundant database queries, thereby improving system efficiency and scalability. The cache can be configured with expiration policies to ensure that outdated data is automatically removed, maintaining the accuracy and freshness of the information provided to users. This approach enhances overall user experience by reducing latency and improving response times.

Although caching is not implemented in the current version of the Real Estate Support Triage Agent, it is an important architectural consideration for performance optimization. Conversational AI systems frequently receive repeated or similar property queries. Without caching, each query requires full processing, including entity extraction, database filtering, and response generation, which increases latency and system workload.

Introducing a caching layer such as Redis between the backend and database would significantly reduce redundant database calls. Frequently requested property results could be temporarily stored and quickly retrieved for identical or similar queries. This would reduce response time and improve overall user experience. Additionally, caching LLM-generated responses for repeated structured queries could lower computational overhead and reduce external API costs.

Proper cache expiration policies would be required to ensure that cached property data remains consistent with the database. Time-based invalidation or version-based cache keys could be implemented to maintain data freshness. By integrating caching into the architecture, the system would achieve improved scalability, reduced server load, and faster response times. The current modular design ensures that a caching layer can be introduced in the future without major structural modifications.

7. Non-Functional Requirements

- Scalability: Supports increasing users
- Availability: Ensures minimal downtime
- Maintainability: Modular structure
- Reliability: Stable operations
- Security: Data protection
- Usability: Easy interaction

7.1. Security Aspects

- OTP-based authentication
- Secure password hashing
- API validation
- Environment variable protection

7.2. Performance Aspects

- Asynchronous API handling
- Optimized database queries
- Containerized deployment
- Scalable infrastructure

8. References

1. FastAPI Documentation. *FastAPI – Modern, Fast Web Framework for Building APIs with Python.*
   Available at: https://fastapi.tiangolo.com/
2. MongoDB Documentation. *MongoDB Official Documentation.*
   Available at: https://www.mongodb.com/docs/
3. Docker Inc. *Docker Documentation – Containerization Platform.*
   Available at: https://docs.docker.com/
4. HashiCorp. *Terraform Documentation – Infrastructure as Code Tool.*
   Available at: https://developer.hashicorp.com/terraform/docs
5. Red Hat. *Ansible Documentation – Automation and Configuration Management.*
   Available at: https://docs.ansible.com/
6. Redis Documentation. *Redis In-Memory Data Store Documentation.*
   Available at: https://redis.io/documentation/
7. LangChain Documentation. *LangChain Framework for LLM Applications.*
   Available at: https://python.langchain.com/
8. Google AI. *Generative AI and Large Language Models Documentation.*
   Available at: https://ai.google.dev/
9. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures (REST).* University of California, Irvine.